

Data Analysis

1. Dataset Quality Check

```
1 def missing_unknown_plot(df):
2     # count missing values (NaN) per column
3     missing_counts = df.isna().sum()
4     # count unknown string vals
5     unknown_counts = (
6         df.astype(str).apply(lambda col: col.str.lower().eq("unknown").sum())
7         if not df.empty
8         else pd.Series(0, index=df.columns)
9     )
10
11     # combine into one Series
12     total_issues = missing_counts + unknown_counts
13     total_issues = total_issues[total_issues > 0].sort_values(ascending=False)
14     if total_issues.empty:
15         print("No missing or 'Unknown' values found. Dataset looks clean!")
16         return
17     plt.figure(figsize=(12, 6))
18     total_issues.plot(kind="bar", color="firebrick")
19     plt.title("Missing/Unknown Values per Column")
20     plt.xlabel("Column")
21     plt.ylabel("Count")
22     plt.xticks(rotation=45, ha="right")
23     plt.tight_layout()
24     plt.show()
```

Function `missing_unknown_plot` visualizes missing and 'Unknown' values per column to assess dataset quality.

```
Dataset summary showing shape, sample row, CWE count, and confirmation of no missing or 'Unknown' values.
```

In order to check the data quality, the distribution of missing values and Unknown values for each column was analyzed. As a result of the analysis, missing values and "Unknown" values were hardly found in the entire dataset. This means that unnecessary or damaged records were effectively removed during the data collection and preprocessing process. Therefore, this dataset has a low risk of bias or decline in data quality due to missing information and is suitable for subsequent machine learning models.

2. Class Distribution

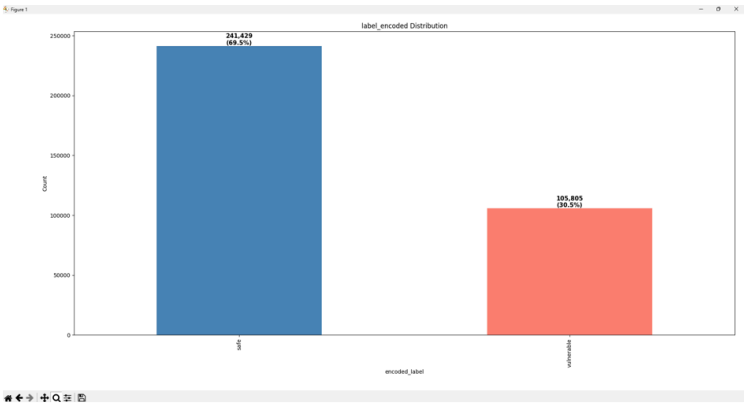


Figure 1: Distribution of safe and vulnerable code samples in the dataset, indicating class imbalance

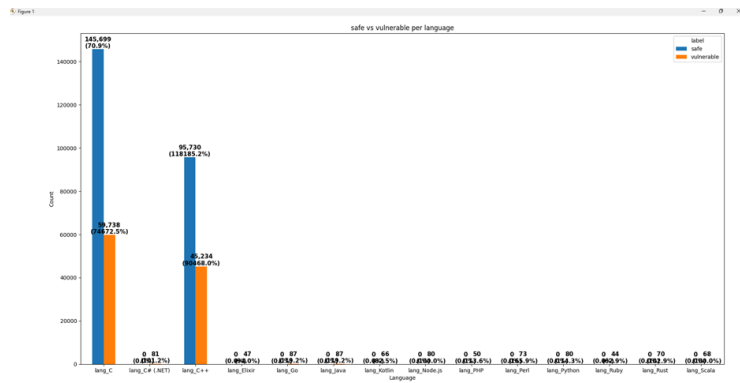


Figure 2: Safe vs Vulnerable code distribution by programming language, showing C and C++ as the dominant languages in the dataset with both classes present.

Analysis of the class distribution of the dataset showed that out of a total of 347,234 samples, 241,429 (69.5%) were safe codes and 105,805 (30.5%) were Vulnerable codes (Figure 1). This shows that the dataset is somewhat unbalanced and that the Safe code is about twice as many as the Vulnerable code.

As a result of looking at the distribution by language together (Figure 2), the main data is composed of C and C++ languages, and Safe code accounts for a higher proportion than Vulnerable code in both languages. In particular, in the C language, Safe code appears to be about 71%, showing a pattern similar to the overall trend of the entire dataset.

This class imbalance poses a risk that the model will be biased toward multiple classes when learning machine learning models. Therefore, in order to improve model performance, complementary measures such as data resampling or undersampling techniques, class weight adjustment, or applying a robust algorithm to the imbalance must be considered.

3. Language Distribution

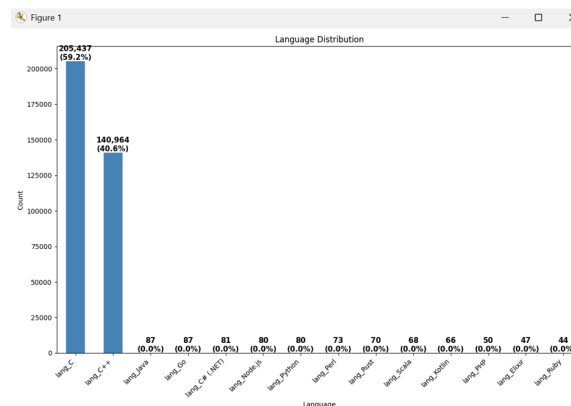


Figure 3: Overall language distribution in the dataset, showing that C (59.2%) and C++ (40.6%) dominate, while other languages contribute less than 0.1%.

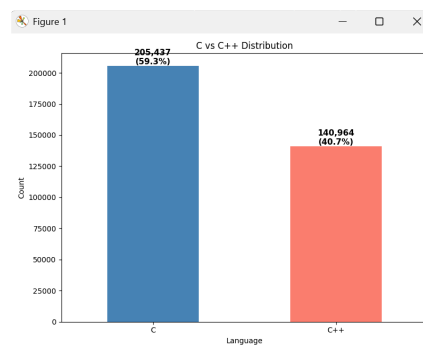


Figure 4: Comparison of C and C++ code samples, highlighting that C (59.3%) slightly outweighs C++ (40.7%) in the dataset.

As a result of analyzing the language distribution of the dataset, the entire data was almost entirely composed of C and C++ codes. Specifically, 205,437 C languages (59.2%) and 140,964 C++ languages (40.6%) were found, and the two languages accounted for the majority of the total dataset. On the other hand, other languages such as Java, Go, Python, C#, and Node.js were less than 100 in number, less than 0.1% of the total proportion.

These results show that the dataset is in fact organized around C and C++, and machine learning model training and analysis should also focus primarily on these two languages.

4. Vulnerability Type (CWE) Distribution

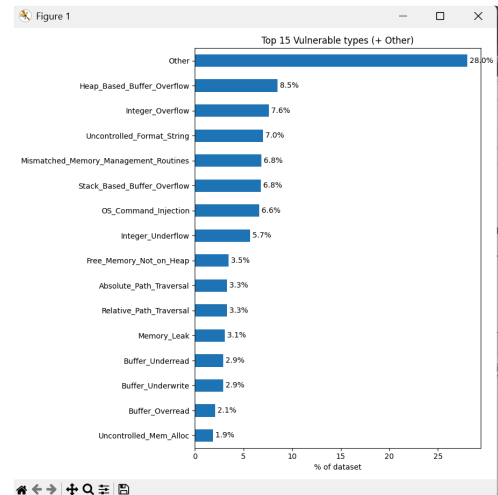


Figure 5: Distribution of the top 15 CWE vulnerability types (+ Other)

As a result of the data analysis, this dataset is almost entirely biased towards C(59%) and C++(41%), and the label distribution is also clearly class unbalanced at Safe 69.5%/Vulnerable 30.5%. There are 220 different CWE types in the total data, but the top 15 CWEs account for approximately 72% of the data (the remaining 28% are formed), especially vulnerabilities related to memory buffers such as heap/stack buffer overflow, integer overflow, format string, and memory management errors were dominant. This distribution suggests that (1) the risk of model performance being concentrated on the most frequent minority classes in the multi-class (per-CWE) classification, (2) the CWE-Token-based features are more decisive than the language-specific differences as the data is in fact C/C++-centric. Therefore, it is best to mitigate the imbalance by class weighting, stratified sampling, oversampling, etc. when designing the model, and make sure to report the precision/recall/F1 and confusion matrix by class in the evaluation. A realistic and stable alternative is to first perform a binary classification of vulnerability vs safety and then classify hierarchical approaches or low-frequency CWEs into semantic groups to further classify only the main CWEs.

5. Feature Analysis (TF-IDF & Tokens)

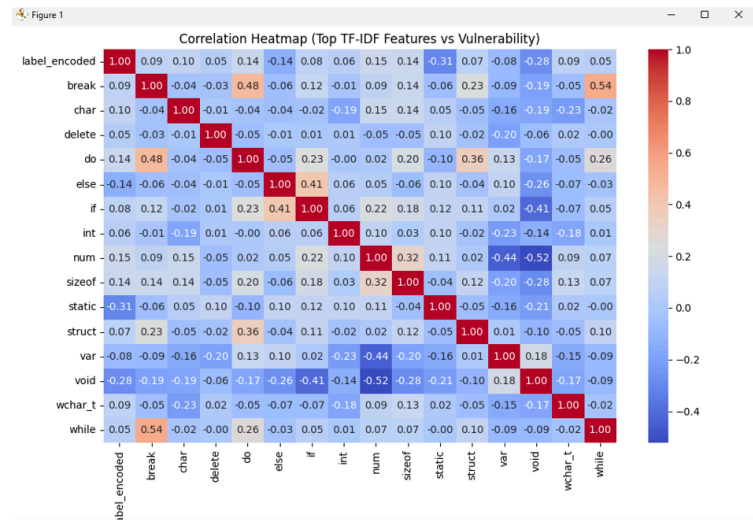


Figure 6: Correlation heatmap of top TF-IDF features and vulnerability label, highlighting token-label relationships and structural token pairs such as `if-else` and `while-break`.

As a result of TF-IDF-based token correlation analysis (Figure), some tokens were clearly associated with vulnerability. For example, keywords such as static (-0.31) and void (-0.28) showed negative correlations, indicating features that appear more frequently in safety codes, while num (+0.15), do (+0.14), and sizeof (+0.14) showed positive correlations with vulnerable codes. In addition, in the correlation between tokens, typical patterns that appear together in conditional and repetitive statements structures, such as if-else (+0.41), do-break (+0.48), and while-break (+0.54) were identified. These results show that certain keywords can be useful features for vulnerability prediction when learning models, and at the same time suggest that token pairs with high correlations can cause multicollinearity problems, so they need to be dealt with properly through feature selection or regulatory techniques.

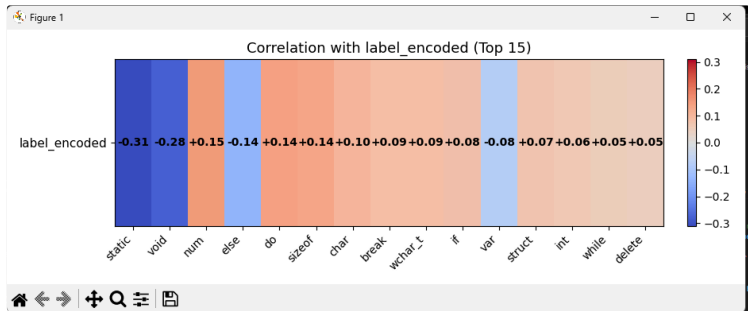


Figure 7: Correlation of the top 15 TF-IDF features with the vulnerability label, showing negative associations

When looking at the correlation between the TF-IDF token and the vulnerability label, some keywords showed a clear tendency. Keywords such as static (-0.31), void (-0.28),... were more strongly linked to the safety code, whereas num (+0.15), do (+0.14), and sizeof (+0.14) showed a positive correlation with the vulnerable code. This suggests that certain tokens can act as a feature that distinguishes them from the safety or vulnerability of the code and can be considered as important predictors when learning the model.

6. Comparison of Token Importance (TF-IDF Sum) Between Safe and Vulnerable Code

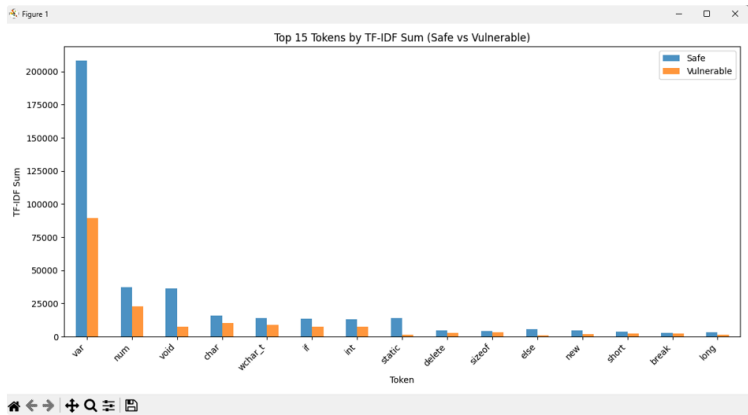


Figure 8: Comparison of the top 15 tokens by TF-IDF sum between Safe and Vulnerable code

This graph shows the results of comparing the Safe code and the Vulcanable code for the top 15 tokens based on the total TF-IDF among tokens extracted from the dataset. The x-axis represents the token and the y-axis represents the sum of the TF-IDF values calculated across each function for the corresponding token. The larger the value, the more frequent the token appears in the functions and is reflected as an important feature by the rarity (IDF). Overall, the fact that the Safe bar is higher overall than the vulnerable code, so the fact that the Safe bar is higher can be interpreted as a result of reflecting the effect of the difference in data amount rather than the importance of the token itself. Nevertheless, in the Safe code, declaration- and data-type related tokens such as var, num, and void occupy a high proportion, while in the vulnerable code, the contribution of these tokens is relatively small. This suggests that Safety Code may tend to exploit more diverse declarations and explicit keywords, meaning that if data imbalances are not corrected during model training, Safe Code characteristics are likely to be over-reflected.

7. Insights and Implications for ML

This dataset has no missing or 'Unknown' values, so it is reliable in terms of quality and is suitable for machine learning models. Data analysis results give several important implications in the process of designing and learning the model. First, the fact that certain words such as static, void, num, and do showed a clear difference in distinguishing between vulnerable and safe codes shows that word-based analysis methods (TF-IDF + machine learning models) can be effective. Second, there is a risk that the model will be trained only toward

Safe because Safe codes account for about 70% of the total dataset, so the ratio differs significantly. To prevent this, it is necessary to use techniques such as how to match the number of data (oversampling/ undersampling) or class weight adjustment. Finally, given that the dataset is almost composed of only C and C++ codes, models tailored to these languages are likely to perform best. In addition, stepwise classification (e.g., first classifying Safe and Vulnerable, then classifying the main CWE types in detail) can be a realistic and stable alternative.